



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from First Principles	
Topic Name	The use of generic types to model and capture large classes of data structures by factorizing common patterns part 1	
Topic Code	-	
Unit/Module No. & Name	8	Polymorphism
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4-5
2.0	Meaning And Definition	6-8
3.0	Nature Or Type	9-11
4.0	Examples And Case Studies	12-15
5.0	Keywords And Touch Vocabulary	16



OBJECTIVE

1. This unit explains generic types, also called generics, in clear and simple academic English for Online MCA learners.
2. It shows how one well-made pattern can represent many data structures such as lists, stacks, queues, trees, and maps.
3. You will learn what a type parameter is and how it acts like a blank space that is filled with a real type later.
4. You will also learn how generics reduce repeated code.
5. You will learn that generics reduce the need for type casting.
6. You will learn that generics reduce run-time type errors.
7. The unit introduces safe flexibility tools such as bounded type parameters.
8. The unit also introduces wildcards so that you can read common library code with confidence.
9. By the end of this unit, you should be able to explain why generics are important.
10. You should also be able to write basic generic code.
11. You should be able to connect generics to the idea of factorizing common patterns in data structure design.

1.0 INTRODUCTION

1.0.1 Why This Topic Matters

Many computer programs store, sort, search, and move data. Students meet lists, stacks, queues, trees, and maps in early courses because these structures are the basic tools of software. Each tool follows a rule. A stack follows last in, first out. A queue follows first in, first out. A list keeps items in order. A map links keys to values. These rules are the same no matter what the items are. The items could be numbers, words, student records, images, or messages. The rule stays the same, but the item type changes, and that change affects how safe your program is.

If we write a new stack class for each item type, we create many copies of the same idea. Copying code makes it harder to test and harder to fix. When you fix one bug, you must remember to fix it in every copy. Copying also increases the chance of errors because people forget small details. In real teams, copied code becomes a maintenance problem. For online students, copied code also makes projects longer than they need to be.

Generics solve this problem by letting a class or method accept a type as an input. This is like accepting a number as an input, but the input is a type name. The idea is called parametric polymorphism. It means one definition can work for many types without changing the logic. A generic stack does the same push and pop steps for any item type. Only the item type varies. The compiler checks the type choices and warns you early when you misuse them.

1.0.2 From Reuse to Factorizing Patterns

The phrase factorizing common patterns means we separate what stays the same from what changes. In data structures, the links, pointers, and steps stay the same, while the element type changes. Generics let us factor out the element type into a parameter, often written as T, E, K, or V. Then one structure can model a large class of related structures. A generic List<T> can represent a list of Student, a list of Course, or a list of String without rewriting the list logic.

This is not only a coding trick. It is also a design mindset. You look at a family of problems and find the shared shape. Then you keep the shared shape in one place and allow the small differences to be supplied later. In software engineering, this helps you build libraries that many people can use. In academic work, it helps you show clean thinking and clean abstraction.

1.0.3 Where You See Generics in Daily Coding

Even if you do not write your own generic class on day one, you already use generics when you use standard libraries. In Java, ArrayList<String>, HashMap<Integer, Student>, and Optional<Course> are common. In C#, you see List<T> and Dictionary<TKey, TValue>. In C++, you see template classes like vector<T>. Many frameworks also use generics for data

access layers, typed responses, and reusable services. That is why an undergraduate student should understand the idea early, not late.

1.0.4 Learning With an Online Mindset

Online MCA learners often study in short sessions and practice with small programs. Generics fit this approach well because you can start with tiny examples, such as a `Box<T>`, and grow toward bigger structures, such as a `Tree<T>`. Each step repeats the same idea: keep the structure stable and let the type vary. When you revise later, the pattern still makes sense, which supports long-term learning.



2.0 MEANING AND DEFINITION

2.0.1 Meaning of Generic Types

A generic type is a class, interface, or method that uses one or more type parameters. A type parameter is a name that stands for a real type, such as Integer, String, or Student, when the code is used. Instead of fixing the element type inside the code, we leave a slot for the type. When we create an object, we fill the slot with a specific type. In Java, we write this with angle brackets, such as `Box<String>` or `List<Student>`.

In simple words, generics mean write once, use with many types, and still stay safe. Safety matters because the compiler can check that you are not mixing types in a way that will break later. Without generics, you might add a Student into a list meant for Course by mistake. With generics, the compiler can stop you and show an error before the program runs.

2.0.2 Definition in Academic Terms

In academic terms, generics are a form of parametric polymorphism where functions or data types are written with parameters that range over types. The code is defined once and then instantiated for each chosen type. In many textbooks, this is explained as writing algorithms over an abstract type and later supplying a concrete type. The key point is that the algorithm logic does not depend on the concrete type. It depends only on operations that the type is guaranteed to have, such as comparison, equality, or simple storage.

Many languages support this idea, including Java, C#, and C++ where it is called templates. Java's approach is designed for backward compatibility. Most generic type checks happen at compile time, and the run-time representation often removes the type parameter details. This is called type erasure. Even with type erasure, the compile-time checks still give strong safety and clearer code.

2.0.3 What Happens Without Generics

To understand the meaning, it helps to remember what coding looked like before generics. In older Java, collection classes stored values as `Object`. `Object` is the parent of most classes, so you could place many kinds of items in one container. But later, when you took an item out, you had to cast it back. Casting is a manual promise by the programmer. If the promise is wrong, the program may crash with a `ClassCastException`. This crash can happen far from the place where the wrong item was added, which makes debugging harder.

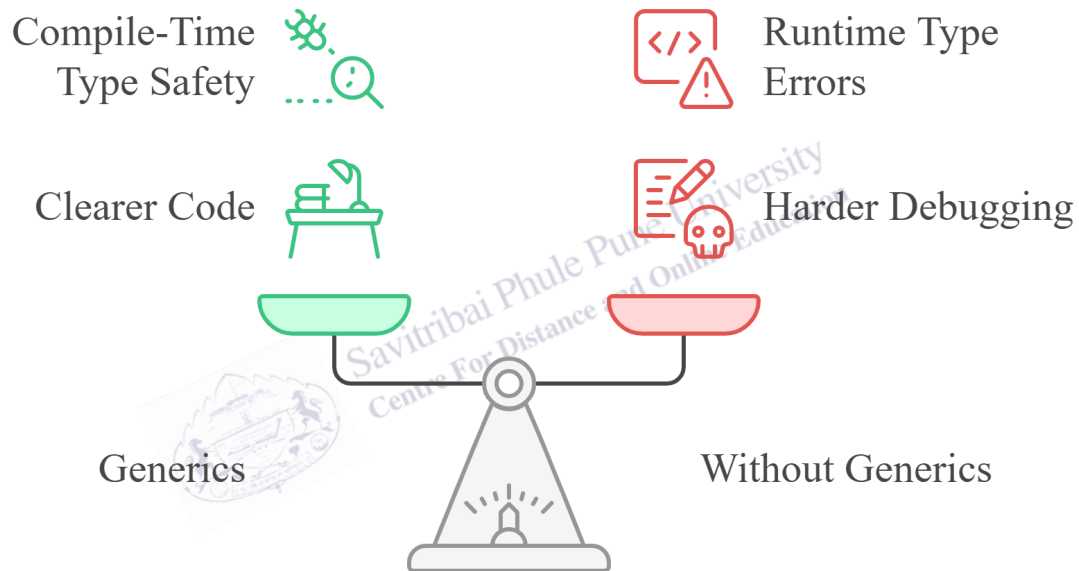
Generics reduce this risk by putting the promise into the type system. The programmer states the element type once, like `List<Student>`. Then the compiler checks every add and every read. This keeps the container consistent and reduces the need for casting.

2.0.4 Key Components

A generic design includes three key pieces. First, a type parameter name, such as T for type, E for element, K for key, or V for value. Second, a place where the parameter is used, such as a field, method input, or return value. Third, an instantiation step, where a user chooses a real type. For example, a generic `Pair<K, V>` has two parameters. When used as `Pair<String, Integer>`, the key is a String and the value is an Integer.

Fig. 1

Generics enhance type safety and code clarity.



Generics improve code quality by enforcing compile-time type safety and clarity. They reduce runtime errors and simplify debugging. Compared to non-generic code, generics balance software design by minimizing casting, improving readability, and helping developers write safer, reusable data structure implementations.

A helpful way to think about this is as a contract. The generic class promises it will store and return items consistently. The user promises to supply items of the declared type. Together, they keep the program safer and clearer.

2.0.5 Relationship to Abstract Data Types

Data structure courses often describe structures as abstract data types. An abstract data type describes what operations exist, not how they are stored. For example, a stack is defined by push, pop, and peek. The definition does not depend on whether the stack uses an array or a linked list. Generics fit this view well. A $\text{Stack}\langle T \rangle$ describes a family of stacks for any type T . The operations are the same, and the implementation can be chosen separately. This supports clean theory and clean code at the same time.



3.0 NATURE OR TYPE

3.0.1 Generic Classes and Interfaces

The most common form of generics is a generic class or interface. This means the whole type carries a type parameter. Examples are `List<E>`, `Stack<T>`, `Set<E>`, and `Map<K, V>`. In a generic list, methods like `add` and `get` use `E`. In a generic map, `put` uses `K` and `V` and `get` returns `V`. When you supply real types, the method signatures become specific. For instance, if you use `Map<String, Integer>`, then `get` returns `Integer`, not `Object`.

This nature of generics supports library design. The library author writes one correct implementation. Many users apply it to many types. The library can be tested once, and the benefit spreads widely.

3.0.2 Generic Methods

A second form is a generic method. A class may not be generic, but one method can be. A simple example is a method that prints arrays of any element type. Another example is a method that finds the maximum of items, as long as items can be compared. In Java, this often uses a bound such as `T extends Comparable<T>`. In simple terms, the method works for all types that can compare themselves.

Generic methods are helpful for algorithms that do not depend on a stored state. They can be placed in utility classes and reused across projects. For students, generic methods are a good practice step before building bigger generic classes.

3.0.3 Bounded Type Parameters

Sometimes a method cannot accept any type. For example, a `sum` method needs numbers, not words. A sorting method needs comparable items, not random objects. Bounded type parameters express this rule. In Java, we write `T extends Number` or `T extends Comparable<T>`. The word `extends` is used even when the bound is an interface. The meaning is that `T` must be a subtype of the bound.

Bounds make generic design more precise. They tell the user what operations are allowed. They also help the compiler provide better error messages when a wrong type is used.

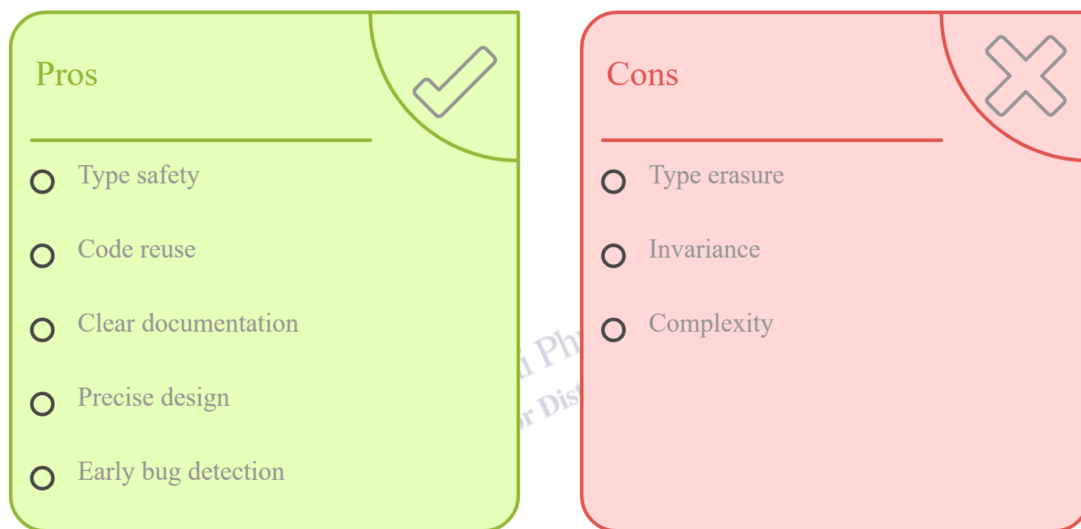
3.0.4 Wildcards and Variance

When you use generics, you also meet a key type relation question: if `Cat` is a subtype of `Animal`, is `List<Cat>` a subtype of `List<Animal>`? In Java, the answer is no. This is called invariance. It avoids unsafe writes. If `List<Cat>` were treated as `List<Animal>`, then someone could add a `Dog`, and the list would no longer be a list of cats.

To handle safe flexibility, Java uses wildcards. A wildcard is written as `?`, meaning some unknown type. You can also bound it. The wildcard `?` extends `Animal` means some type that is `Animal` or a subtype of `Animal`. This is useful when you mainly read values. The wildcard `?` super `Cat` means some type that is `Cat` or a supertype of `Cat`. This is useful when you mainly add cats. Teachers often summarize this as PECS: Producer Extends, Consumer Super. The phrase is short, but it matches real API design in Java.

Fig. 2

Java Generics



Java Generics provide type safety, code reuse, clear documentation, precise design, and early bug detection. However, they introduce limitations such as type erasure, invariance issues, and added complexity, which can make generic code harder to understand for beginners.

3.0.5 Type Erasure as a Feature of Java Generics

Java implements generics with type erasure. The compiler uses type parameters to check safety, but the run-time bytecode usually does not keep full type parameter details. For example, `List<String>` and `List<Integer>` both become `List` at run time. This kept Java compatible with older code written before generics existed. It also explains why you cannot write `new T()` inside a generic class, and why you cannot create new arrays of `T` in a direct way.

Type erasure does not remove the value of generics. It shifts the main benefit to compile time. For Online MCA students, the key lesson is that good design uses the compiler as a helper. Generics are a strong example of letting tools catch bugs early.

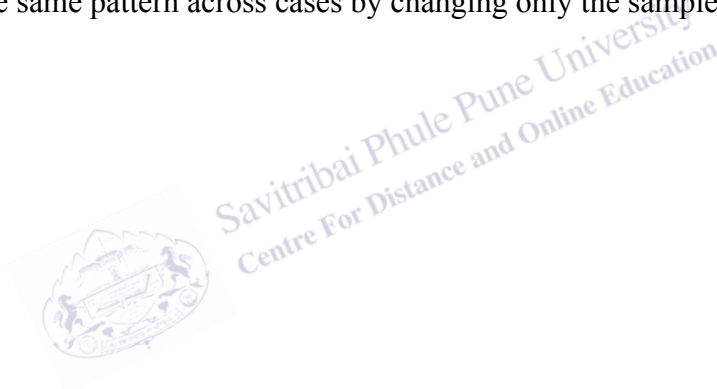
3.0.6 Why Invariance Protects Data Structures

Invariance can feel strict, but it protects data structures from silent corruption. Imagine a method that takes `List<Animal>` and adds a Dog. If you pass a `List<Cat>` into that method, the method would add a Dog into the cat list. The error might not show until later, when you assume every item is a Cat. By making `List<Cat>` not a subtype of `List<Animal>`, Java prevents this unsafe call. Wildcards then provide controlled flexibility when you need it.

3.0.7 Generics in Documentation

Generics improve documentation because type information appears in the API itself. When a method returns `List<Student>`, you know what the list contains. You do not need extra comments to guess the element type. This matters in large code bases where many people read the same methods. In academic work, clear signatures also help graders and classmates understand your intent, which supports smoother group projects. This small habit can prevent confusion when you revisit code months later.

Generics can also support safer unit tests. With explicit types, test helpers can build `List<T>` values and reuse the same pattern across cases by changing only the sample data.



4.0 EXAMPLES AND CASE STUDIES

4.0.1 Example: A Generic Box

Consider a simple class named `Box<T>`. It stores one item and lets you set and get that item. The class does not care if the item is a string, a student record, or a course code. The only rule is that the type is consistent for each box. When you create `Box<String>`, the compiler ensures you only place strings inside. When you create `Box<Student>`, it ensures you only place `Student` objects inside.

This small example shows the core pattern: structure plus operations remain the same, while the stored type changes. It also shows how generics reduce casting. Without generics, you might store an `Object` and cast it back later. If you cast wrong, you get a run-time error. With generics, a wrong type is blocked earlier.

4.0.2 Example: A Generic List for Student Names

Now think about a list that stores student names for an attendance app. If you declare `List<String> names`, then each add must be a string, and each get returns a string. This seems simple, but it matters. It keeps the rest of the code clean. You do not need to check the type each time. You can also use string methods right away because the compiler knows the type. This reduces clutter and makes the code easier to read.

In an undergraduate project, clarity is not only about style. Clear code is easier to mark, easier to debug, and easier to extend. Generics help you reach that clarity.

4.0.3 Example: A Generic Stack for Undo

A common use of stacks is an undo feature. Imagine a text editor for students writing assignments. Each action, such as insert, delete, or replace, can be modeled as an `Action` object. When the user performs an action, the program pushes it onto a stack. When the user presses undo, the program pops the last action and reverses it.

With a generic `Stack<Action>`, the program remains clean. You can be sure the stack holds only `Action` objects. If a programmer tries to push a `String` by mistake, the compiler catches it. The same stack logic could also support other tools, like undo in a drawing app, where the actions are `DrawAction` objects. Again, one pattern supports many cases through a type parameter.

4.0.4 Example: A Generic Pair and Map

A map stores key-value pairs. In Java, `Map<K, V>` is generic. It can represent `Map<String, Integer>` for roll numbers, `Map<Integer, Student>` for lookup by ID, or `Map<String, List<Course>>` for course lists by department. The map logic stays the same: compute a position

for the key, store the pair, and retrieve the value later. The type parameters help the compiler ensure that keys and values match the expected types.

This example shows how generics model a large class of structures. A map is not just one structure. It is a family of structures for all possible key and value types. The common pattern is factorized as `Map<K, V>`, and each real use fills K and V as needed.

4.0.5 Case Study: Generic Repository in a University Portal

Consider a simple university portal. It stores students, courses, and results. Each entity needs basic actions: create, read, update, and delete. If a developer writes a separate repository class for Student, Course, and Result, the code can become repetitive. The same database steps appear again and again, with only the entity type changing. Repetition can also hide bugs. For example, one repository might close a database connection correctly, while another forgets. This can cause resource leaks and slow performance.

A generic repository can reduce repetition. A class `Repository<T>` can define methods such as `save(T item)`, `findById(int id)`, and `delete(T item)`. Then you can use `Repository<Student>` and `Repository<Course>` without rewriting the core logic. This approach encourages consistent error handling, consistent logging, and consistent testing. It also supports future growth. If the portal adds a new entity, such as Faculty, the same repository pattern can serve it with little new code.

This case connects to software engineering goals. Reuse lowers cost, reduces defects, and improves clarity for teams. Generics help achieve reuse while keeping type safety, which is essential in shared code bases.

4.0.6 Case Study: Bounded Generics for Grading Rules

Now consider grading logic. A program might compute averages for numeric scores. If you want a method that sums items, you should not accept any type. You should accept only numeric types. Bounded generics can express this. A method like `sum(List<? extends Number> scores)` can read numbers from the list and sum them. It can accept a list of Integer, Double, or Float. The bound expresses a rule: the element type must extend Number.

This design avoids unsafe assumptions. If someone tries to pass a `List<String>`, the compiler rejects it. At the same time, the method remains reusable across many numeric types. This matters in academic code because students often change data types during testing. Bounds let you keep the same method even when you switch from Integer to Double.

4.0.7 Case Study: Wildcards and a Safe Copy Method

Suppose you build a library method that copies data from one list to another. If you want to copy from a list of integers into a list of numbers, you need a type-safe signature. A typical pattern is

`copy(List<? super T> dest, List<? extends T> src)`. The source produces items, so it uses `extends`. The destination consumes items, so it uses `super`. This design is common in standard libraries because it balances flexibility and safety.

In student projects, you may not write such complex signatures at first. Still, understanding the idea helps you read library methods and avoid common errors, such as trying to add items to a list declared with `? extends`. The main lesson is simple: design your types based on how data flows, and let the compiler guide you.

4.0.8 Case Study: Generic Tree Nodes for Hierarchies

Many applications store hierarchical data, such as a file system, an organization chart, or course prerequisites. A tree node often stores a value plus links to children. The links are the same for any tree, but the value type can change. With a generic `TreeNode<T>`, the same structure can store `TreeNode<String>` for folder names, `TreeNode<Course>` for course objects, or `TreeNode<Integer>` for numeric keys. This helps you reuse traversal methods, such as pre-order and post-order, with many kinds of payloads.

This case also shows that generics can be combined with interfaces. If you want to sort children, you can bound `T` to `Comparable<T>`. The bound expresses a clear rule: the node value must be comparable if sorting is required.

4.0.9 Common Student Mistakes and How to Avoid Them

Students often face four common mistakes. First, they use raw types like `List` without angle brackets. Raw types remove many safety checks. Second, they try to add items to a list declared with an `extends` wildcard. This fails because the compiler cannot be sure of the exact element type. Third, they confuse `extends` with `super`. Remember that `extends` is mainly for reading and `super` is mainly for writing. Fourth, they expect generics to exist fully at run time. Because of type erasure, some run-time tasks require extra work, such as passing a `Class` object or using factory methods.

The simplest habit is this: always declare the most specific generic type you can and avoid raw types unless you are working with old code. When in doubt, read the method signature and ask, “Am I producing items, consuming items, or both?” That question often leads you to the right wildcard choice.

4.0.10 Summary of Learning

Across these examples, the theme repeats. Generics separate structure from stored type. They help you build one data structure pattern that works for many types. They prevent many mistakes before the program runs. They also support clean interfaces, where rules are visible in the method signature. In an Online MCA program, these skills matter for coding assessments,

project work, and internships. They also prepare you for reading framework code, because modern frameworks rely on generics to express safe design.

1. Generics let one data structure pattern work for many types.
2. Type parameters such as T, E, K, and V act as placeholders for real types.
3. Compile-time checking reduces the need for casting and lowers run-time errors.
4. Wildcards provide flexible reading and writing rules in APIs.
5. Bounded types limit generics to safe families like Number or Comparable.
6. Type erasure explains why some generic details are not present at run time.
7. Real systems use generics in collections, repositories, and utility methods.
8. Understanding generics improves your ability to read standard library code.
9. Good generic design makes code easier to test and maintain.
10. Practice by rewriting simple stacks, queues, and trees using generics.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple meaning	Example use
Generic	Works with many types	Generic stack
Type parameter	Placeholder for a type	T in Stack
Parametric polymorphism	Same logic for many types	printArray(T[])
Wildcard	Unknown type marker	List<?>
Upper bound	Limit using extends	? extends Number
Lower bound	Limit using super	? super Integer
Invariance	No subtype relation in generics	List not List
Type erasure	Type info removed at run time	List becomes List
Cast	Convert Object to a type	(Student) obj
PECS	Rule for wildcards	Producer Extends

